

**Predicción del nivel de congestión en las estaciones del Metro de Medellín mediante
árboles de decisión**

Julian Vega Joya y Alejandro Mora

Ingeniería de Software, Corporación Universitaria Iberoamericana

Inteligencia Artificial (24082026_C1_202634)

Sandra Bautista

9 de octubre de 2026

Predicción del nivel de congestión en las estaciones del Metro de Medellín mediante árboles de decisión

Introducción

En las actividades anteriores del curso trabajamos sobre el sistema de transporte masivo de Medellín y construimos un sistema que busca la mejor ruta entre dos estaciones. Esa solución deja una pregunta abierta: la ruta más corta no siempre es la mejor si una de sus estaciones está colapsada en el momento del viaje. Para responderla necesitábamos que el sistema anticipara qué tan llena va a estar cada estación, y ese es el problema que abordamos en esta actividad.

El aprendizaje automático ofrece herramientas para este tipo de predicción. En particular, el aprendizaje supervisado permite que un programa aprenda a partir de ejemplos cuya respuesta ya se conoce y luego generalice a casos nuevos (Mitchell, 1997). Entre sus métodos, los árboles de decisión se destacan porque producen modelos que una persona puede leer como un conjunto de reglas (Palma Méndez y Marín Morales, 2008), algo valioso cuando el resultado tiene que explicarse a quien opera el sistema.

Este documento presenta el modelo que desarrollamos: un árbol de decisión que predice si una estación de las líneas A y B del Metro de Medellín tendrá congestión baja, media o alta en una hora determinada. Primero se exponen los objetivos y el marco teórico; después se describen las fuentes de datos, la construcción del dataset, la implementación en Python y las pruebas; y al final se presentan los resultados, los enlaces de la entrega y las conclusiones.

Objetivos

Objetivo general

Desarrollar un modelo de aprendizaje supervisado basado en árboles de decisión que prediga el nivel de congestión de las estaciones del Metro de Medellín a partir de variables conocidas de antemano.

Objetivos específicos

1. Identificar y describir fuentes de datos públicas relacionadas con la demanda del sistema de transporte masivo de Medellín.
2. Construir un dataset de muestra con la estructura necesaria para entrenar el modelo, dado que no existe una fuente pública con la afluencia por estación y por hora.
3. Implementar en Python el entrenamiento, la optimización y la evaluación de un árbol de decisión.
4. Verificar mediante pruebas automáticas y manuales la calidad de los datos y el comportamiento del modelo.

Marco teórico

Aprendizaje supervisado

Un programa aprende de la experiencia cuando su desempeño en una tarea mejora a medida que acumula esa experiencia (Mitchell, 1997). En el aprendizaje supervisado la experiencia consiste en ejemplos etiquetados: pares formados por una descripción del caso, mediante un conjunto de atributos, y la respuesta correcta para ese caso. A partir de ellos el algoritmo construye una función que asigna una respuesta a casos que no ha visto (Russell y Norvig, 2021). Cuando la respuesta es una categoría, como en nuestro problema, la tarea se denomina clasificación.

Árboles de decisión

Un árbol de decisión clasifica un caso mediante una secuencia de preguntas sobre sus atributos. Cada nodo interno evalúa un atributo, cada rama corresponde a un posible resultado de esa evaluación y cada hoja asigna una clase. Recorrer el árbol desde la raíz hasta una hoja equivale a aplicar una regla del tipo "si se cumplen estas condiciones, entonces la clase es esta", por lo que el modelo puede expresarse como un conjunto de reglas de decisión (Palma Méndez y Marín Morales, 2008).

Los algoritmos más conocidos para construir árboles son ID3 (Quinlan, 1986) y su sucesor C4.5 (Quinlan, 1993), además de CART (Breiman et al., 1984). Todos siguen una estrategia voraz de arriba hacia abajo: en cada nodo eligen el atributo que mejor separa las clases, dividen los ejemplos según ese atributo y repiten el proceso en cada subconjunto hasta que los nodos son suficientemente puros.

Entropía y ganancia de información

Para decidir qué atributo separa mejor las clases, ID3 y C4.5 usan la entropía, una medida de la incertidumbre de un conjunto que proviene de la teoría de la información (Shannon, 1948). Para un conjunto S con clases de proporción $p(i)$, la entropía es $H(S) = -\sum p(i) \cdot \log_2 p(i)$. Vale cero cuando todos los ejemplos pertenecen a la misma clase y es máxima cuando las clases están igualmente repartidas. La ganancia de información de un atributo es la reducción de entropía que se obtiene al dividir el conjunto según ese atributo; el algoritmo elige, en cada nodo, el atributo con mayor ganancia (Quinlan, 1986).

Sobreajuste y poda

Si un árbol crece sin restricciones puede llegar a clasificar perfectamente los ejemplos de entrenamiento memorizando sus particularidades, incluido el ruido, y aun así fallar con casos nuevos. Este fenómeno se conoce como sobreajuste (Mitchell, 1997). La solución es la poda, que limita el crecimiento del árbol o elimina ramas que no mejoran su capacidad de generalizar (Palma Méndez y Marín Morales, 2008; Quinlan, 1993). En la práctica se controla con parámetros como la profundidad máxima o el número mínimo de ejemplos que debe tener cada hoja.

Evaluación del modelo

Un modelo debe evaluarse con datos distintos de los que usó para aprender. Por eso los datos se dividen en un conjunto de entrenamiento y otro de prueba, y la configuración del modelo se elige con validación cruzada sobre el conjunto de entrenamiento, una técnica que da estimaciones confiables del desempeño (Kohavi, 1995). Las métricas habituales en clasificación son la exactitud, o proporción de aciertos; la precisión y la exhaustividad de cada clase; el F1, que combina las dos anteriores; y la matriz de confusión, que muestra qué clases se confunden entre sí (Hastie et al., 2009).

Desarrollo

Fuentes de datos

Buscamos fuentes públicas sobre la demanda del sistema. El Metro de Medellín publica en su portal de datos abiertos la afluencia de pasajeros por línea, día y hora (Metro de Medellín, s.f.), y su informe a la Asociación Latinoamericana de Metros y Subterráneos reporta cerca de 713 000 usos diarios en las líneas A y B (Metro de Medellín, 2017). También identificamos el catálogo georreferenciado de estaciones, un archivo GTFS con los recorridos y la red de pluviómetros del Valle de Aburrá. El detalle de cada fuente está en el documento de descripción de los datos.

Ninguna de estas fuentes contiene la afluencia por estación y por hora, que es el nivel de detalle que necesita el modelo. Por eso aplicamos la segunda opción del enunciado de la actividad y construimos un dataset con una muestra de los datos.

Construcción del dataset

El script `data/generar_dataset.py` genera 29 176 registros: una fila por cada una de las 27 estaciones de las líneas A y B, cada hora de operación y cada día entre el 2 de marzo y el 26 de abril de 2026. Las estaciones, el calendario con sus festivos y el orden de magnitud de la demanda son reales. El número de pasajeros es simulado a partir del tipo de zona de la estación, el perfil horario, el día de la semana, la lluvia, los eventos masivos y un componente aleatorio. La etiqueta se obtiene del índice de ocupación: menos de 0,45 es Bajo, de 0,45 a 0,80 es Medio y desde 0,80 es Alto.

Las columnas de pasajeros y de índice de ocupación se excluyen del entrenamiento, porque la etiqueta se calcula a partir de ellas y su uso sería una fuga de información.

Implementación del modelo

El modelo está en `src/modelo_congestion.py` y usa la biblioteca `scikit-learn` (Pedregosa et al., 2011). Sigue seis pasos:

1. Carga el dataset con `pandas` y verifica que no falten columnas ni haya valores nulos.
2. Separa las nueve variables predictoras de la etiqueta.
3. Divide los datos en 80 % para entrenamiento y 20 % para prueba, de forma estratificada.
4. Construye un flujo que convierte las variables categóricas en columnas binarias y entrena un árbol de decisión con criterio de entropía, el mismo de ID3 y C4.5.
5. Busca la mejor configuración entre 35 combinaciones de profundidad máxima y tamaño mínimo de hoja, con validación cruzada de cinco particiones.
6. Evalúa el árbol elegido con el conjunto de prueba y guarda las métricas, las reglas y los gráficos.

El archivo `src/predecir.py` permite consultar el modelo desde la línea de comandos para una estación, día y hora concretos.

Pruebas

Verificamos el componente de tres maneras: con la evaluación estadística en el conjunto de prueba, con 27 pruebas automáticas en pytest (9 sobre la calidad de los datos y 18 sobre el comportamiento del modelo) y con consultas manuales. Las 27 pruebas automáticas se aprobaron. El detalle está en el documento de pruebas realizadas.

Resultados

La Tabla 1 resume el desempeño del modelo.

Tabla 1

Desempeño del árbol de decisión

Métrica	Valor
Registros (entrenamiento / prueba)	29 176 (23 340 / 5836)
Configuración elegida	Sin límite de profundidad y mínimo 10 ejemplos por hoja
Exactitud en el conjunto de prueba	84,0 %
Línea base (predecir siempre la clase más frecuente)	47,0 %
Exactitud en validación cruzada	83,7 %
Exactitud en entrenamiento	85,9 %
F1 macro	0,830
Errores graves (confundir Bajo con Alto)	0 de 5836
Pruebas automáticas aprobadas	27 de 27

El árbol acierta el 84,0 % de los casos que nunca vio, frente al 47,0 % de la línea base, y la diferencia entre entrenamiento y prueba es pequeña, lo que indica que la poda controló el sobreajuste. La Figura 1 muestra que el modelo nunca confunde una estación con congestión baja con una de congestión alta: todos sus errores ocurren entre niveles vecinos.

Figura 1

Matriz de confusión del conjunto de prueba

Matriz de confusión (conjunto de prueba)				
Nivel real	Bajo	Medio	Alto	
	2.470 90%	271 10%	0 0%	
	337 15%	1.733 79%	132 6%	
	0 0%	193 22%	700 78%	
		Bajo	Medio	Alto
		Nivel que predijo el modelo		

Las variables que más pesan en las decisiones del árbol son la hora (40,4 %) y la estación (22,9 %), seguidas por el tipo de día y el tipo de zona. El efecto de los eventos masivos es pequeño en el total, porque son poco frecuentes, pero cambia la predicción cuando ocurren: la estación Estadio un sábado a las 19:00 pasa de congestión baja a alta si hay partido.

Enlaces de la entrega

La Tabla 2 presenta los enlaces al repositorio y al video. La docente fue agregada como colaboradora del repositorio para que pueda revisar el código y dejar comentarios.

Tabla 2*Enlaces de la entrega*

Elemento	Enlace
Repositorio Git	https://github.com/Julianibero/Inteligencia-artificial-SANDRA-BAUTISTA-24082026_C1_202634-/tree/Actividad-3---M%C3%A9todos-de-aprendizaje-supervisado
Video explicativo	https://www.youtube.com/watch?v=slo6jjVrCpQ

Los cinco elementos solicitados se encuentran en el repositorio, como lo muestra la Tabla 3.

Tabla 3*Ubicación de los entregables en el repositorio*

N.º	Entregable	Ubicación
1	Archivos de fuentes de datos	data/afluencia_metro_medellin.csv , data/estaciones.csv , data/generar_dataset.py
2	Código fuente en Python	src/modelo_congestion.py , src/predecir.py
3	Documento con la descripción de los datos	docs/Descripcion_de_los_datos.pdf
4	Documento con las pruebas realizadas	docs/Pruebas_realizadas.pdf , tests
5	Video	https://www.youtube.com/watch?v=slo6jjVrCpQ

Conclusiones

El árbol de decisión resultó adecuado para el problema. Alcanzó una exactitud de 84,0 % con datos nuevos, muy cerca del máximo que permite el ruido de la muestra, y sus decisiones pueden leerse como reglas comprensibles, por ejemplo: si es un día laboral, en la hora pico de la tarde y en una zona de empleo, la congestión será alta. Esa capacidad de explicación es lo que distingue a los árboles de decisión de otros métodos (Palma Méndez y Marín Morales, 2008).

La poda fue necesaria. Sin restricciones, el árbol mejoraba en entrenamiento mientras empeoraba en validación, el sobreajuste que describe la literatura (Mitchell, 1997). Exigir un mínimo de diez ejemplos por hoja corrigió ese comportamiento.

Las pruebas mostraron que una exactitud global alta puede ocultar fallas en los casos poco frecuentes: en una primera versión, con pocos días de partido, el modelo no había aprendido el efecto de los eventos. Ampliar la muestra lo corrigió, y dejamos ese caso como prueba automática.

La principal limitación es que la afluencia por estación es simulada, de modo que los resultados demuestran que el método funciona pero no describen la operación real del Metro. El paso siguiente es obtener la afluencia real por estación y conectar esta predicción con el buscador de rutas de las actividades anteriores, para recomendar rutas que eviten las estaciones congestionadas.

Referencias

- Breiman, L., Friedman, J. H., Olshen, R. A., y Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.
- Hastie, T., Tibshirani, R., y Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.^a ed.). Springer.
<https://doi.org/10.1007/978-0-387-84858-7>
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. En *Proceedings of the 14th International Joint Conference on Artificial Intelligence* (Vol. 2, pp. 1137–1145). Morgan Kaufmann.
- Metro de Medellín. (s.f.). *Pasajeros movilizados: afluencia* [Conjunto de datos]. Datos Abiertos Colombia.
<https://www.datos.gov.co/Transporte/Pasajeros-Movilizados-Afluencia/9dbb-kbnv>
- Metro de Medellín. (2017). *Metro de Medellín* [Informe]. Asociación Latinoamericana de Metros y Subterráneos.
<https://alamys.org/wp-content/uploads/2021/04/Metro-de-Medellin.pdf>
- Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
- Palma Méndez, J. T., y Marín Morales, R. L. (Coords.). (2008). *Inteligencia artificial: Métodos, técnicas y aplicaciones*. McGraw-Hill.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
<https://doi.org/10.1007/BF00116251>
- Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. Morgan Kaufmann.
- Russell, S., y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.^a ed.). Pearson.

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>